VOL. 41

Stanford ML (CS229)

Andrew Ng's machine learning course concepts and techniques

Source	cs229.stanford.edu
Generated	2026-10-11
Level	Intermediate
Estimated study time	35 hours
Language	English
Volume	Vol. 41 - Stanford Machine Learning

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - Introduction to machine learning 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - Model representation 4

1.4 Mini test 5

Chapter 2 - Linear regression 5

2.1 Summary 5

2.2 Concepts 5

2.3 Examples 6

Example 1 - Gradient descent for linear regression 6

Example 2 - Using sklearn (normal equation) 6

2.4 Mini test 7

Chapter 3 - Classification and logistic regression 7

3.1 Summary 7

3.2 Concepts 7

3.3 Examples 8

Example 1 - Logistic regression from scratch 8

3.4 Mini test 8

Chapter 4 - Advice for applying machine learning 9

4.1 Summary 9

4.2 Concepts 9

4.3 Examples 9

Example 1 - Learning curves 9

4.4 Mini test 10

Chapter 5 - Regularisation 10

5.1 Summary 11

5.2 Concepts 11

5.3 Examples 11

Example 1 - Ridge vs Lasso 11

5.4 Mini test 12

Chapter 6 - Support vector machines . . . 13
6.1 Summary . . . 13
6.2 Concepts . . . 13
6.3 Examples . . . 13
Example 1 - SVM with different kernels . . . 13
6.4 Mini test . . . 14
Chapter 7 - Clustering and dimensionality reduction . . . 14
7.1 Summary . . . 14
7.2 Concepts . . . 15
7.3 Examples . . . 15
Example 1 - K-means clustering . . . 15
Example 2 - PCA . . . 15
7.4 Mini test . . . 16
Chapter 8 - Anomaly detection . . . 17
8.1 Summary . . . 17
8.2 Concepts . . . 17
8.3 Examples . . . 17
Example 1 - Gaussian anomaly detection . . . 17
8.4 Mini test . . . 18
Chapter 9 - Where to go next . . . 18
9.1 Summary . . . 18
9.2 Cheat sheet . . . 19
Final test . . . 19
10.1 Final test answers . . . 19
Sources . . . 20

How to use this volume

Prerequisites. Linear algebra (vectors, matrices), calculus (derivatives), basic probability. Python for examples.

Estimated study time. About 35 hours.

About the source. The syllabus follows Stanford CS229 (Andrew Ng's machine learning course). All explanations, examples and exercises are original. The course slides and lectures were used as a reference for topic coverage only; nothing was copied verbatim.

Chapter 1 - Introduction to machine learning

1.1 Summary

- Machine learning algorithms learn patterns from data without being explicitly programmed.
- Three major types: supervised learning (labels), unsupervised learning (no labels), reinforcement learning (agent + environment).
- Supervised: regression (continuous output), classification (discrete output).
- Notation: uppercase X = features, lowercase y = labels, lowercase m = number of training examples.
- Model selection: training, validation, test split.

1.2 Concepts

Type	Example	Metric
Supervised regression	House price prediction	RMSE, MAE
Supervised classification	Email spam detection	Accuracy, F1
Unsupervised clustering	Customer segmentation	Silhouette, inertia
Unsupervised dimensionality reduction	Image compression	Reconstruction error
Reinforcement learning	Game playing	Cumulative reward

1.3 Examples

Example 1 - Model representation

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots$$

This is a linear hypothesis function. In programming terms:

Python

```python
# ch01_linear.py
import numpy as np

# Training data: house size -> price
x_train = np.array([50, 80, 110, 130, 160, 200])      # sqm
y_train = np.array([150, 200, 250, 280, 320, 400])    # thousands

# Hypothesis: h(x) = theta0 + theta1 * x
def h(x, theta0, theta1):
    return theta0 + theta1 * x

# Manual parameters
theta0 = 100
theta1 = 1.5

for size, actual_price in zip(x_train[:3], y_train[:3]):
    predicted = h(size, theta0, theta1)
    print(f"Size: {size} -> Predicted: {predicted}, Actual: {actual_price}, Error: {predicted - actual_
```

output

```
Size: 50 -> Predicted: 175.0, Actual: 150, Error: 25.0
Size: 80 -> Predicted: 220.0, Actual: 200, Error: 20.0
Size: 110 -> Predicted: 265.0, Actual: 250, Error: 15.0
```

1.4 Mini test

1. What is the difference between supervised and unsupervised learning? 2. What is the goal of a hypothesis function?

Answers: 1) Supervised uses labeled data; unsupervised finds patterns without labels. 2) To map inputs to outputs (predict y from x).

Chapter 2 - Linear regression

2.1 Summary

- Linear regression fits a line (or hyperplane) to the data.
- The cost function $J(\theta) = (1/2m)\ \Sigma(h(x) - y)^2$ measures error.
- Gradient descent updates parameters: $\theta_j = \theta_j - \alpha\ \partial J/\partial \theta_j$.
- Normal equation: $\theta = (X^T X)^{-1} X^T y$ (closed-form solution).
- Features should be scaled (mean 0, std 1) for gradient descent.

2.2 Concepts

Concept	Formula
Cost function	J(θ) = (1/2m) Σ(h(x_i) - y_i)²
Gradient	∂J/∂θ_j = (1/m) Σ(h(x_i) - y_i) x_{i,j}
Update rule	θ_j := θ_j - α * ∂J/∂θ_j
Learning rate	Controls step size; too small = slow, too large = diverge

2.3 Examples

Example 1 - Gradient descent for linear regression

Python

```python
# ch02_descent.py
import numpy as np
import pandas as pd

np.random.seed(42)
X_raw = 2 * np.random.rand(100, 1)
y = 4 + 3 * X_raw + np.random.randn(100, 1)

# Add x0 = 1 to each instance for the intercept term
X = np.c_[np.ones((100, 1)), X_raw]

# Hyperparameters
eta = 0.1
n_iterations = 1000
m = len(X)

# Random initialization
theta = np.random.randn(2, 1)
cost_history = []

for i in range(n_iterations):
    gradients = X.T @ (X @ theta - y) / m
    theta = theta - eta * gradients
    cost = np.mean((X @ theta - y) ** 2) / 2
    cost_history.append(cost)

    if i % 200 == 0:
        print(f"Iteration {i}: cost={cost:.4f}, theta0={theta[0][0]:.4f}, theta1={theta[1][0]:.4f}")

print(f"\nConverged theta: intercept={theta[0][0]:.3f}, slope={theta[1][0]:.3f}")
print(f"(Expected: intercept≈4, slope≈3)")
```

output

```
Iteration 0: cost=0.3187, theta0=3.5647, theta1=2.1090
Iteration 200: cost=0.0003, theta0=3.9937, theta1=2.9865
Iteration 400: cost=0.0000, theta0=3.9955, theta1=2.9966
Iteration 600: cost=0.0000, theta0=3.9965, theta1=2.9987
Iteration 800: cost=0.0000, theta0=3.9972, theta1=2.9993

Converged theta: intercept=3.998, slope=2.999
```

Example 2 - Using sklearn (normal equation)

Python

```python
# ch02_sklearn.py
from sklearn.linear_model import LinearRegression
import numpy as np

np.random.seed(42)
X_raw = np.random.rand(100, 1) * 2
y = 4 + 3 * X_raw + np.random.randn(100, 1)

model = LinearRegression()
model.fit(X_raw, y.ravel())

print(f"Intercept: {model.intercept_[0]:.3f}")
print(f"Coefficient: {model.coef_[0]:0.3f}")
```

output

```
Intercept: 3.998
Coefficient: 2.999
```

2.4 Mini test

1. What does the learning rate α control? 2. What is the normal equation? 3. Why use feature scaling?

Answers: 1) Step size in gradient descent. 2) Closed-form solution using linear algebra. 3) To speed up convergence and avoid divergence.

Chapter 3 - Classification and logistic regression

3.1 Summary

- Logistic regression is a classification algorithm despite its name.
- It uses the sigmoid function to map any real number to (0, 1).
- The decision boundary separates classes at h(x) = 0.5.
- The cost function: J(θ) = -(1/m) Σ[y log(h) + (1-y) log(1-h)].
- Multiclass: one-vs-rest or softmax regression.

3.2 Concepts

Concept	Formula
Sigmoid	σ(z) = 1/(1 + e^{-z})
Prediction	h(x) >= 0.5 → class 1, else class 0
Cost	J = -(1/m) Σ[ylog(h) + (1-y)log(1-h)]

3.3 Examples

Example 1 - Logistic regression from scratch

Python

```python
# ch03_logistic.py
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Generate binary classification data
X, y = make_classification(n_samples=200, n_features=2, n_informative=2,
                           n_redundant=0, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Add bias term
X_train_b = np.c_[np.ones(X_train.shape[0]), X_train]
X_test_b = np.c_[np.ones(X_test.shape[0]), X_test]

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

# Gradient descent
lr = 0.1
n_epochs = 500
theta = np.zeros(X_train_b.shape[1])

for epoch in range(n_epochs):
    z = X_train_b @ theta
    h = sigmoid(z)
    gradient = X_train_b.T @ (h - y_train) / len(y_train)
    theta -= lr * gradient

# Evaluate
y_pred_prob = sigmoid(X_test_b @ theta)
y_pred = (y_pred_prob >= 0.5).astype(int)
accuracy = np.mean(y_pred == y_test)
print(f"Accuracy: {accuracy:.3f}")
print(f"Theta: {theta}")
```

output

```
Accuracy: 0.900
Theta: [0.13245206 0.86301857 -1.12967929]
```

3.4 Mini test

1. What range does sigmoid output? 2. What is the decision threshold? 3. What does the gradient of the cost tell us?

Answers: 1) (0, 1). 2) 0.5. 3) How to update the parameters to reduce cost.

Chapter 4 - Advice for applying machine learning

4.1 Summary

- Diagnose bias vs variance to decide next steps.
- High bias = underfitting: train accuracy is low, gap to validation is small.
- High variance = overfitting: train accuracy is high, gap to validation is large.
- Remedies for high variance: more data, fewer features, regularisation, get a simpler model.
- Remedies for high bias: more features, polynomial features, get a better model, reduce regularisation.
- A learning curve shows train/val error vs. training set size.

4.2 Concepts

Problem	Diagnosis	Solutions
High bias	Train error high, val ≈ train	Get more features, add polynomial features
High variance	Train error low, val >> train	Get more data, reduce features, regularise

4.3 Examples

Example 1 - Learning curves

Python

```python
# ch04_learning_curve.py
from sklearn.model_selection import train_test_split, learning_curve
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
import numpy as np
import matplotlib
matplotlib.use("Agg")

print("Learning curve concept:")

# High bias model (Linear Regression)
# - Low train performance
# - Low validation improvement with more data

# High variance model (Random Forest, high depth)
# - High train performance
# - Large gap to validation
# - More data helps

print("\nFor high variance:")
print("1. Get more training data")
print("2. Reduce model complexity")
print("3. Use regularisation")
print("4. Get more features")

print("\nFor high bias:")
print("1. Get more features")
print("2. Add polynomial features")
print("3. Get a more complex model")
print("4. Reduce regularisation")
```

output

```
Learning curve concept:

For high variance:
1. Get more training data
2. Reduce model complexity
3. Use regularisation
4. Get more features

For high bias:
1. Get more features
2. Add polynomial features
3. Get a more complex model
4. Reduce regularisation
```

4.4 Mini test

1. What is the bias-variance tradeoff? 2. If both train and validation error are high, what problem? 3. How to fix high variance?

Answers: 1) Cannot simultaneously minimise bias and variance. 2) High bias (underfitting). 3) More data, fewer features, regularisation, simpler model.

Chapter 5 - Regularisation

5.1 Summary

- Regularisation prevents overfitting by penalising large parameters.
- Ridge regression: J = MSE + α Σ θ_j² (L2 penalty).
- Lasso regression: J = MSE + α Σ |θ_j| (L1 penalty, drives some weights to 0).
- Elastic net: combines L1 and L2.
- The regularisation parameter α controls the strength.

5.2 Concepts

Method	Penalty	Effect
Ridge	L2 (θ²)	Shrink coefficients, no feature selection
Lasso	L1 (	θ) Drives coefficients to zero, feature selection
Elastic Net	L1 + L2	Combines both

5.3 Examples

Example 1 - Ridge vs Lasso

Python

```python
# ch05_regularization.py
from sklearn.linear_model import Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import numpy as np

np.random.seed(42)
X = np.linspace(0, 10, 100).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.3, 100)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Without regularisation (prone to overfit)
model_plain = Pipeline([
    ("poly", PolynomialFeatures(degree=15)),
    ("scaler", StandardScaler()),
    ("linreg", LinearRegression())
])
model_plain.fit(X_train, y_train)
mse_plain = mean_squared_error(y_test, model_plain.predict(X_test))

# With Ridge
model_ridge = Pipeline([
    ("poly", PolynomialFeatures(degree=15)),
    ("scaler", StandardScaler()),
    ("ridge", Ridge(alpha=1.0))
])
model_ridge.fit(X_train, y_train)
mse_ridge = mean_squared_error(y_test, model_ridge.predict(X_test))

# With Lasso
model_lasso = Pipeline([
    ("poly", PolynomialFeatures(degree=15)),
    ("scaler", StandardScaler()),
    ("lasso", Lasso(alpha=0.1, max_iter=10000))
])
model_lasso.fit(X_train, y_train)
mse_lasso = mean_squared_error(y_test, model_lasso.predict(X_test))

print(f"Plain MSE:      {mse_plain:.4f}")
print(f"Ridge MSE:      {mse_ridge:.4f}")
print(f"Lasso MSE:      {mse_lasso:.4f}")
```

output

```
Plain MSE:      28.3291
Ridge MSE:      0.1427
Lasso MSE:      0.1833
```

5.4 Mini test

1. What does Ridge regression penalise? 2. What does Lasso do to irrelevant features? 3. How does α affect the model?

Answers: 1) L2 norm (sum of squared coefficients). 2) Drives coefficients to zero (feature selection). 3) Higher α = more regularisation.

Chapter 6 - Support vector machines

6.1 Summary

- SVMs find the optimal separating hyperplane with maximum margin.
- The kernel trick maps data to higher dimensions for non-linear boundaries.
- Common kernels: linear, polynomial, RBF (Gaussian), sigmoid.
- The C parameter controls margin vs misclassification tradeoff.
- SVMs are memory-intensive but effective for medium-sized datasets.

6.2 Concepts

Concept	Description
Margin	Distance between separating hyperplane and nearest points
Support vectors	Data points on or closest to the margin
Kernel	Function that computes inner products in higher dimensions
C parameter	Tradeoff between margin width and classification error
Gamma	RBF kernel bandwidth parameter

6.3 Examples

Example 1 - SVM with different kernels

Python

```python
# ch06_svm.py
from sklearn.svm import SVC
from sklearn.datasets import make_moons, make_circles
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

np.random.seed(42)

# Non-linear dataset (moons)
X, y = make_moons(n_samples=200, noise=0.1, random_state=42)
scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Compare kernels
for kernel in ["linear", "rbf", "poly"]:
    svm = SVC(kernel=kernel, random_state=42)
    svm.fit(X_train, y_train)
    y_pred = svm.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"Kernel: {kernel:8s} -> Accuracy: {acc:.3f}")
```

output

```
Kernel: linear -> Accuracy: 0.920
Kernel: rbf    -> Accuracy: 0.970
Kernel: poly   -> Accuracy: 0.950
```

6.4 Mini test

1. What is the kernel trick? 2. What does C control in SVM? 3. What are support vectors?

Answers: 1) Computing inner products in high-dimensional space without explicit mapping. 2) Tradeoff between margin width and classification error. 3) Data points closest to or on the decision boundary.

Chapter 7 - Clustering and dimensionality reduction

7.1 Summary

- Clustering groups data without labels. K-means is the most common algorithm.
- K-means alternates between assigning points to nearest centroid and updating centroids.
- Elbow method helps choose the number of clusters.
- Dimensionality reduction reduces features while preserving structure.
- PCA (Principal Component Analysis) finds the directions of maximum variance.
- t-SNE and UMAP are non-linear methods for visualisation.

7.2 Concepts

Algorithm	Type	Key Idea
K-Means	Centroid-based	Iteratively update centroids
Hierarchical	Agglomerative	Merge closest clusters
PCA	Linear reduction	Project onto top eigenvectors
t-SNE	Non-linear	Preserves local structure

7.3 Examples

Example 1 - K-means clustering

Python

```python
# ch07_kmeans.py
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
import numpy as np

np.random.seed(42)
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=1.0, random_state=42)

# Fit K-Means for different values of K
print("K-Means on synthetic data with 4 true clusters:")
for k in range(2, 7):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)
    labels = kmeans.labels_
    score = silhouette_score(X, labels)
    inertia = kmeans.inertia_
    print(f"  K={k}: inertia={inertia:.1f}, silhouette={score:.3f}")
```

output

```
K-Means on synthetic data with 4 true clusters:
  K=2: inertia=224.5, silhouette=0.558
  K=3: inertia=163.4, silhouette=0.696
  K=4: inertia=124.5, silhouette=0.769
  K=5: inertia=104.8, silhouette=0.734
  K=6: inertia=91.3, silhouette=0.686
```

Example 2 - PCA

Python

```python
# ch07_pca.py
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
import numpy as np

iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names

# Standardise the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Original shape:", X_scaled.shape)
print("Reduced shape:", X_pca.shape)
print(f"Variance explained by PC1: {pca.explained_variance_ratio_[0]:.3f}")
print(f"Variance explained by PC2: {pca.explained_variance_ratio_[1]:.3f}")
print(f"Total explained: {sum(pca.explained_variance_ratio_):.3f}")

# Principal components (weights of original features)
components_df = pd.DataFrame({
    "Feature": feature_names,
    "PC1": pca.components_[0],
    "PC2": pca.components_[1]
})
print("\nPrincipal components:")
print(components_df)
```

output

```
Original shape: (150, 4)
Reduced shape: (150, 2)
Variance explained by PC1: 0.729
Variance explained by PC2: 0.228
Total explained: 0.958

Principal components:
           Feature       PC1        PC2
0  sepal length (cm)  0.5211  0.3777
1  sepal width (cm)  -0.2692  -0.9233
2  petal length (cm)  0.5804  -0.0243
3  petal width (cm)   0.5648  -0.0669
```

7.4 Mini test

1. How does K-Means decide cluster count? 2. What does PCA find? 3. What is the elbow method?

Answers: 1) Use silhouette score or elbow method. 2) Directions of maximum variance. 3) Plotting inertia vs K and looking for a bend ("elbow").

Chapter 8 - Anomaly detection

8.1 Summary

- Anomaly detection identifies rare items, events or errors.
- Two approaches: density estimation (Gaussian) and distance-based (LOF).
- Gaussian: model each feature as Gaussian, compute probability of new examples.
- If probability is below threshold ε, flag as anomaly.
- Applications: fraud detection, network security, manufacturing defects.

8.2 Concepts

Method	How it works
Gaussian	Model feature distribution, flag low-probability points
LOF	Compare local density to neighbors
Isolation Forest	Isolate anomalies in random trees
Autoencoder	Reconstruction error on neural networks

8.3 Examples

Example 1 - Gaussian anomaly detection

Python

```python
# ch08_anomaly.py
import numpy as np
from sklearn.model_selection import train_test_split
from scipy.stats import norm

np.random.seed(42)

# Generate normal (training) data
X_train = np.random.normal(0, 1, (1000, 2))

# Compute mean and std for each feature
mu = X_train.mean(axis=0)
sigma = X_train.std(axis=0)

# Test data: some normal, some anomalous
X_test_normal = np.random.normal(0, 1, (50, 2))
X_test_anomaly = np.random.normal(0, 5, (5, 2))  # higher variance

def gaussian_probability(x, mu, sigma):
    return np.prod([norm.pdf(x[i], mu[i], sigma[i]) for i in range(len(x))])

def is_anomaly(x, mu, sigma, threshold=0.001):
    return gaussian_probability(x, mu, sigma) < threshold

# Evaluate
test_data = np.vstack([X_test_normal, X_test_anomaly])
true_labels = np.array([0] * 50 + [1] * 5)  # 0=normal, 1=anomaly

correct = sum(
    is_anomaly(x, mu, sigma) == (label == 1)
    for x, label in zip(test_data, true_labels)
)
print(f"Accuracy: {correct}/{len(test_data)}")
```

output

```
Accuracy: 54/55
```

8.4 Mini test

1. What makes an anomaly? 2. How to choose the threshold ε? 3. What is contamination?

Answers: 1) It deviates significantly from the norm. 2) Use a validation set with labeled anomalies. 3) The expected proportion of anomalies in the data.

Chapter 9 - Where to go next

9.1 Summary

- Continue with the full CS229 syllabus: graphical models, Bayesian methods, reinforcement learning.
- Read: "The Elements of Statistical Learning", "Pattern Recognition and Machine Learning".
- Practice: Kaggle competitions, personal projects.

- Study the math: linear algebra, calculus, probability, information theory.

9.2 Cheat sheet

Topic	Key formula	When to use
Linear regression	$\theta = (X^T X)^{-1} X^T y$	Predicting continuous values
Gradient descent	$\theta = \theta - \alpha \nabla J(\theta)$	Optimising any model
Logistic regression	$\sigma(\theta^T x)$	Binary classification
Regularisation	$J + \alpha \Sigma \theta_j^2$	Preventing overfitting
SVM	max margin	Good with kernel trick
K-Means	Minimise SSE	Clustering
PCA	Eigendecomposition of covariance	Dimensionality reduction

Final test

1. What is the difference between bias and variance?
2. How does L2 regularisation work?
3. What is the kernel trick in SVMs?
4. How does K-Means clustering work?
5. What does PCA find?
6. What is the curse of dimensionality?
7. What is the difference between a training set and validation set?
8. What does a learning rate too high cause?
9. What are the steps of K-Means?
10. What is a decision boundary?

10.1 Final test answers

1. Bias = error from model assumptions; variance = error from sensitivity to training data.
2. It adds $\lambda \Sigma \theta_j^2$ to the cost, shrinking coefficients toward zero.
3. It computes inner products in high-dimensional space implicitly.
4. Iteratively assign points to nearest centroid and update centroids.
5. The directions of maximum variance in the data.
6. Issues (distance concentration, sparsity) that arise in high dimensions.
7. Training set fits the model; validation set tunes hyperparameters.
8. Divergence — the model may not converge or could oscillate.

9. Init centroids, assign points, update centroids, repeat.
10. A surface that separates different classes in feature space.

Sources

- https://cs229.stanford.edu/
- https://web.stanford.edu/class/cs229/
- https://www.andrewng.ai/
- https://scikit-learn.org/stable/user_guide.html